Version 1.3

January 2026

DATA STRUCTURES

MODULE 3 - STACK and QUEUE

Author:

MUHAMMAD ILHAM PERDANA. S.Tr.T., M.T.

AHMAD NUR MU'MININ

WAHYU ANDIKA

INFORMATICS LABORATORY

University of Muhammadiyah Malang

TABLE OF CONTENTS

TABLE OF CONTENTS **2**
INTRODUCTION **3**
OBJECTIVES 3
MODULE TARGETS 3
PREPARATION 3
KEYWORDS 3
REFERENCES **3**
CONTENTS **4**
1. Stack **4**
1.1 Definition of Stack 4
1.3 Basic Stack Operations 5
1.4 Stack Implementation 6
2. Queue **8**
2.1 Definition of Queue 8
2.2 Characteristics of Queue 8
2.3 Basic Queue Operations 9
2.4 Queue Implementation 9
CODELAB **13**
Exercise 13
DEMO **16**
Task 1 16
Task 2 17
Assessment **18**
Grading Rubric 18
Grading Scale 18

INTRODUCTION

OBJECTIVES

1. Students are able to understand and explain the fundamental concepts of Java Stack and Queue.

MODULE TARGETS

1. Students are able to understand : Implementing concept about Stack & Queue.

PREPARATION

1. Muslim students may recite the following prayer before starting the lesson

رَضِيْتُ بِاللهِ رَبًّا وَبِالإِسْلَامِ دِيْنًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُوْلًا. رَبِّ زِدْنِيْ عِلْمًا وَارْزُقْنِيْ فَهْمًا

2. Laptop or personal computer
3. Java Development Kit
4. Java Runtime Environment
5. IDE (Intellij IDEA, Eclipse, Netbeans, etc.)
6. Strong motivation and commitment to learning

KEYWORDS

Stack, LinkedList, Queue

REFERENCES

https://www.scaler.com/topics/java/stack-and-queue-in-java/.

https://www.educative.io/blog/data-structures-stack-queue-java-tutorial.

https://www.geeksforgeeks.org/queue-using-stacks/.

https://www.geeksforgeeks.org/java/java-program-to-implement-shunting-yard-algorithm/.

Note: These references may contain slight variations in explanation or examples.Focus on understanding the core concepts and apply them to the case study provided in this module.

CONTENTS

1. Stack

1.1 Definition of Stack

A stack can be analogized as a pile of objects, where data items are placed on top of one another. Each element in the stack can only be added or removed from the top position. Data that is located in the middle or at the bottom of the stack cannot be accessed directly. To reach those elements, all data above them must be removed first. This structure reflects a strict access rule, making the stack suitable for situations where the most recently added data must be processed first.

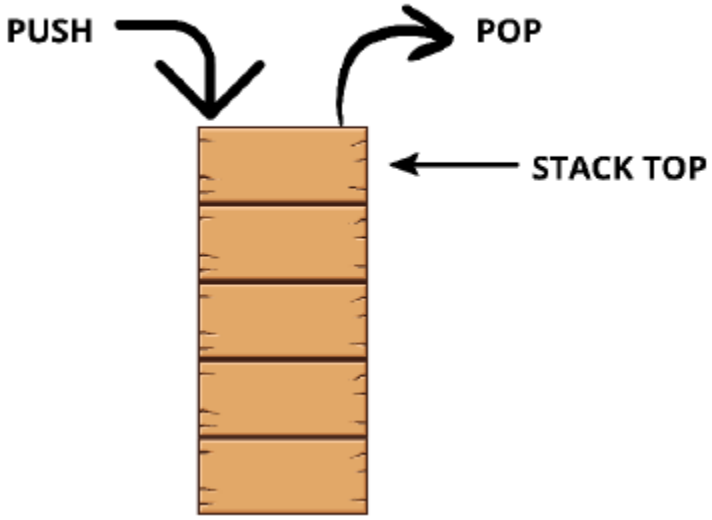


Figure 1. Stack Illustration

1.2 Characteristics of Stack

The stack has a highly restricted access pattern. All operations are performed only at the top of the stack, and no direct access is allowed to elements below it. This limitation ensures that data is processed in a well-defined order.

Stack follows the LIFO (Last-In, First-Out) principle, meaning the last data element inserted into the stack will be the first one to be removed. This characteristic makes stacks particularly useful for managing temporary data, such as execution history or undo operations.

Characteristic	Description
LIFO (Last In, First Out)	The last data entered into the Stack will be the first data removed. In a ticketing system, the last ticket ordered will be canceled first.
One-Way Access	The Stack has only one access point, which is the TOP. Data cannot be retrieved from the middle or bottom.
No Random Access	Stack elements cannot be accessed directly using an index like in an Array or List.
Ordered	The order of data is determined by the time the data entered the Stack.
Suitable for History	Stacks are ideal for storing order history, ticket cancellations, and undo features.

1.3 Basic Stack Operations

All stack operations are performed at the top of the stack. There are three primary operations that define how a stack works.

Method	Description
`push(data)`	Adds data to the top of the Stack. Example: storing the history of new ticket orders.
`pop()`	Deletes and returns the data at the top of the Stack. Example: canceling the last ticket reservation.
`peek()`	Views the data at the top without deleting it. Example: checking the last ticket reserved.
`isEmpty()`	Checks whether the Stack is empty or not. Used before performing `pop()` to avoid errors.
`isFull()`	Checks whether the Stack is full (specifically for Array-based Stacks)

1.4 Stack Implementation

Using a Stack Library

```java
package com.test;

// File: StackLibraryExample.java
import java.util.Stack;

public class StackLibraryExample {
    public static void main(String[] args) {
        Stack<String> riwayatTiket = new Stack<>();

        // Push ()
        riwayatTiket.push("Tiket KA-001");
        riwayatTiket.push("Tiket KA-002");
        riwayatTiket.push("Tiket KA-003");

        System.out.println("Riwayat Pemesanan: " + riwayatTiket);

        // Peek
        System.out.println("Tiket Terakhir: " + riwayatTiket.peek());

        // Pop ()
        System.out.println("Dibatalkan: " + riwayatTiket.pop());

        System.out.println("Sisa Riwayat: " + riwayatTiket);
    }
}

```

Output

```
❯ /usr/bin/env /usr/lib/jvm/java-17-temurin/bin/java -XX:+Sho
202T110254Z-1-001/Modul/1/modul1/target/classes com.test.Stack
Riwayat Pemesanan: [Tiket KA-001, Tiket KA-002, Tiket KA-003]
Tiket Terakhir: Tiket KA-003
Dibatalkan: Tiket KA-003
Sisa Riwayat: [Tiket KA-001, Tiket KA-002]
```

Using a Stack without a Library

```java
class StackManual {
    private String[] data;
    private int top;
    private int kapasitas;

    public StackManual(int kapasitas) {
        this.kapasitas = kapasitas;
        data = new String[kapasitas];
        top = -1;
    }

    // Push
    public void push(String tiket) {
        if (top == kapasitas - 1) {
            System.out.println("Stack penuh!");
            return;
        }
        data[++top] = tiket;
        System.out.println("Pesan tiket: " + tiket);
    }

    // Pop
    public void pop() {
        if (top == -1) {
            System.out.println("Stack kosong!");
            return;
        }
        System.out.println("Batalkan tiket: " + data[top--]);
    }

    // Tampilkan Stack
    public void tampilkan() {
        System.out.print("Riwayat: ");
        for (int i = 0; i <= top; i++) {
            System.out.print(data[i] + " ");
        }
        System.out.println();
    }
}

public class StackManualExample {
    public static void main(String[] args) {
        StackManual stack = new StackManual(5);

        stack.push("Tiket KA-001");
        stack.push("Tiket KA-002");
        stack.push("Tiket KA-003");

        stack.tampilkan();

        stack.pop();
        stack.tampilkan();
    }
}
```

2. Queue

2.1 Definition of Queue

A queue is a collection of data in which elements can only be added on one side, called the rear (or tail), and elements can only be removed from the opposite side, called the front (or head). This structure restricts how data can be accessed, so elements cannot be inserted or removed freely from arbitrary positions.

A queue operates using two clearly defined ends: the head, where elements are removed, and the tail, where elements are added. Unlike a stack, which has only one access point, a queue has two distinct access points. A queue follows the FIFO (First-In, First-Out) principle, meaning that the first data element inserted into the queue will be the first one removed.

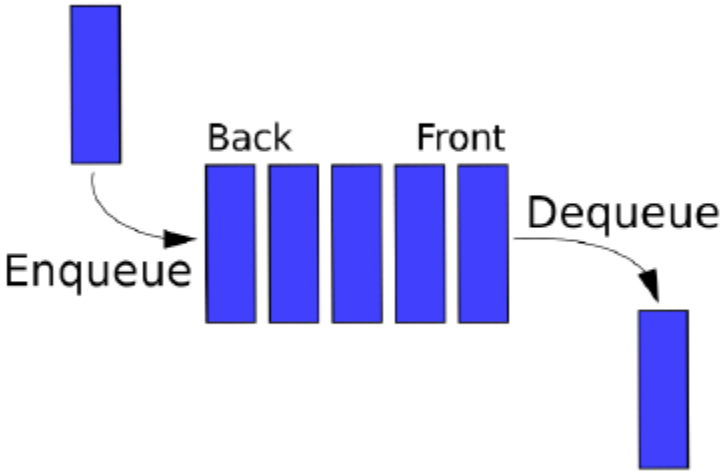


Figure 2. Queue Illustration

In everyday life, a queue can be analogized to a ticket-purchasing line at a railway station. Passengers who arrive earlier are served first. If a new passenger arrives to buy a ticket, they are placed at the very end of the line. The passenger at the last position will be served last and, in some situations, may risk not getting a ticket if availability runs out.

2.2 Characteristics of Queue

A queue has the main characteristic of an orderly and fair data flow. Data insertion is performed only at the rear (*tail*), while data removal is performed only at the front (*head*). Elements located in the middle of the queue cannot be accessed or modified directly.These characteristics ensure that the order of service strictly follows the order of arrival. Therefore, a queue is highly suitable for service-oriented systems, such as railway ticketing systems, where each passenger must be served according to the sequence in which they enter the system.

2.3 Basic Queue Operations

Below are important methods in the Queue Interface often used in implementations using the Java library. This table helps practitioners understand the function of each method conceptually before entering the code.

Method	Description
`add(element)`	Adds an element to the back (tail) of the queue. If the capacity is full, this method will throw an exception.
`offer(element)`	Inserts an element into the queue. Safer than add() because it does not throw an exception when the queue is full, but instead returns false.
`peek()`	Views the element at the head of the queue without removing it. Returns null if the queue is empty.
`element()`	Similar to peek(), but throws a NoSuchElementException if the queue is empty.
`poll()`	Removes and returns the element at the head of the queue. Returns null if the queue is empty.
`remove()`	Removes and returns the element at the head of the queue. Throws an exception if the queue is empty.

2.4 Queue Implementation

A Queue implementation can be carried out using two main approaches: by using Java's built-in library or by constructing the queue structure manually without relying on library support.In the library-based implementation, Java provides the Queue interface, which is commonly implemented using LinkedList.

This approach simplifies queue management, allowing programmers to focus on application logic without dealing with internal data structure details.In a manual implementation, a queue can be built using a Linked List structure with the help of a Node class.

This structure uses two main references, head and tail, to manage enqueue and dequeue operations. The manual approach is especially important for helping students understand the fundamental workings of a queue and realize that a queue is essentially a controlled data flow governed by FIFO rules.

1. **Queue implementation using the Java library**

```java
import java.util.LinkedList;
import java.util.Queue;

public class QueueLibraryExample {
    public static void main(String[] args) {
        Queue<String> ticketQueue = new LinkedList<>();

        ticketQueue.add("Passenger A");
        ticketQueue.add("Passenger B");
        ticketQueue.add("Passenger C");

        System.out.println("Queue: " + ticketQueue);
        System.out.println("Served: " + ticketQueue.poll());
        System.out.println("Remaining Queue: " + ticketQueue);
    }
}
```

Output

```
❯ /usr/bin/env /usr/lib/jvm/java-17-temurin/bin
202T110254Z-1-001/Modul/1/modul1/target/classes
Queue: [Passenger A, Passenger B, Passenger C]
Served: Passenger A
Remaining Queue: [Passenger B, Passenger C]
```

2. **Queue implementation without using the Java library**

   1. **Make Node Class**

```java
class Node {
    String data;
    Node next;

    Node(String data) {
        this.data = data;
        this.next = null;
    }
}

```

2. Creating the Queuelist Class

```java
class QueueList {
    private Node head, tail;

    public QueueList() {
        head = tail = null;
    }

    // Enqueue
    public void enqueue(String data) {
        Node newNode = new Node(data);

        if (tail == null) {
            head = tail = newNode;
            return;
        }
        tail.next = newNode;
        tail = newNode;
    }

    // Dequeue
    public void dequeue() {
        if (head == null) {
            System.out.println("Queue is empty!");
            return;
        }
        System.out.println("Served: " + head.data);
        head = head.next;

        if (head == null) {
            tail = null;
        }
    }

    public void display() {
        Node temp = head;
        System.out.print("Queue: ");
        while (temp != null) {
            System.out.print(temp.data + " ");
            temp = temp.next;
        }
        System.out.println();
    }
}
```

3. Creating the Main Class

```java
public class QueueManualMain {
    public static void main(String[] args) {
        QueueList queue = new QueueList();

        queue.enqueue("Passenger A");
        queue.enqueue("Passenger B");
        queue.enqueue("Passenger C");

        queue.display();
        queue.dequeue();
        queue.display();
    }
}
```

Output

```
❯ cd /home/mipow/Downloads/drive-download-20251
tionMessages -cp /home/mipow/Downloads/drive-dow
Queue: Passenger A Passenger B Passenger C
Served: Passenger A
Queue: Passenger B Passenger C
```

CODELAB

In this CODELAB, students are provided with a complete Java program that demonstrates the basic usage of Stack (LIFO) and Queue (FIFO) in a simple context. Students are required to fill the code and understand the logic. Output expected :

```
Initial Service Queue:
[Andi, Titan, Rapi]
Serving passenger: Andi
Queue after serving:
[Titan, Rapi]
Next passenger to serve: Titan

Transaction History:
[Transaction-1, Transaction-2, Transaction-3]
Undo last transaction: Transaction-3
Transaction History after undo:
[Transaction-1, Transaction-2]
Current top transaction: Transaction-2
Transaction history is not empty. Size: 2
```

Exercise

1. Please complete the implementation code for the Stack and Queue data structures using Java's library clearly and correctly, so that you can understand the program output and deepen your understanding of the concepts.

```java
package com.test;

import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;

public class StackQueueCodelab {
    public static void main(String[] args) {

        // ************************************************************
        // BAGIAN 1: QUEUE DEMONSTRATION (FIFO)
        // ************************************************************
        // Queue digunakan untuk antrian pelayanan di stasiun
        // Penumpang yang datang pertama akan dilayani pertama
        // ************************************************************

        // TODO 1: Inisialisasi Queue untuk antrian pelayanan
        // Clue: Queue adalah interface, implementasinya menggunakan LinkedList
        // Sintaks: Queue<TipeData> namaVariabel = new LinkedList<>();
        Queue<String> serviceQueue = _______;
        // Lengkapi inisialisasi Queue

        // TODO 2: Menambahkan penumpang ke antrian menggunakan add()
        // Clue: Gunakan method add(element) untuk menambahkan ke antrian
        // Sintaks: namaQueue.add("data");
        //
        // Tambahkan 3 penumpang: Andi, Titan, Rapi
        serviceQueue._______("Andi");
        // Lengkapi method untuk menambah ke antrian
        serviceQueue._______("Titan");
        // Lengkapi method untuk menambah ke antrian
        _______._______("Rapi");
        // Lengkapi nama Queue dan method

        // Menampilkan antrian awal
        System.out.println("Initial Service Queue:");
        System.out.println(serviceQueue);

        // TODO 3: Melayani penumpang (mengambil dari depan antrian) menggunakan poll()
        // Clue: Gunakan method poll() untuk mengambil dan menghapus elemen pertama
        // Sintaks: String hasil = namaQueue.poll();
        // poll() mengembalikan null jika queue kosong
        // Alternatif: remove() - throws exception jika kosong
        String servedPassenger = serviceQueue._______();
        // Lengkapi method untuk mengambil dari antrian
        System.out.println("Serving passenger: " + servedPassenger);

        // Menampilkan antrian setelah pelayanan
        System.out.println("Queue after serving:");
        System.out.println(serviceQueue);
```

```java
        // TODO 4: Melihat penumpang di depan antrian tanpa menghapus menggunakan peek()
        // Clue: Gunakan method peek() untuk melihat elemen pertama tanpa menghapus
        // Sintaks: String first = namaQueue.peek();
        String nextPassenger = serviceQueue._______();
        // Lengkapi method untuk melihat elemen pertama
        System.out.println("Next passenger to serve: " + nextPassenger);

        System.out.println();

        // ************************************************************
        // BAGIAN 2: STACK DEMONSTRATION (LIFO)
        // ************************************************************

        // TODO 5: Inisialisasi Stack untuk riwayat transaksi
        // Clue: Gunakan class Stack untuk membuat stack
        // Sintaks: Stack<TipeData> namaVariabel = new Stack<>();
        Stack<String> transactionHistory = _______;
        // Lengkapi inisialisasi Stack

        // TODO 6: Menambahkan transaksi ke stack menggunakan push()
        // Clue: Gunakan method push(element) untuk menambahkan ke stack
        // Sintaks: namaStack.push("data");
        //
        // Tambahkan 3 transaksi: Transaction-1, Transaction-2, Transaction-3
        transactionHistory._______("Transaction-1");
        // Lengkapi method untuk menambah ke stack
        transactionHistory._______("Transaction-2");
        // Lengkapi method untuk menambah ke stack
        _______._______("Transaction-3");
        // Lengkapi nama Stack dan method

        // Menampilkan riwayat transaksi
        System.out.println("Transaction History:");
        System.out.println(transactionHistory);

        // TODO 7: Membatalkan transaksi terakhir (undo) menggunakan pop()
        // Clue: Gunakan method pop() untuk mengambil dan menghapus elemen teratas
        // Sintaks: String hasil = namaStack.pop();
        // pop() throws EmptyStackException jika stack kosong
        String lastTransaction = transactionHistory._______();
        // Lengkapi method untuk mengambil dari stack
        System.out.println("Undo last transaction: " + lastTransaction);

        // Menampilkan riwayat setelah undo
        System.out.println("Transaction History after undo:");
        System.out.println(transactionHistory);

        // TODO 8: Melihat transaksi terakhir tanpa menghapus menggunakan peek()
        // Clue: Gunakan method peek() untuk melihat elemen teratas tanpa menghapus
        // Sintaks: String top = namaStack.peek();
        String topTransaction = transactionHistory._______();
        // Lengkapi method untuk melihat elemen teratas
        System.out.println("Current top transaction: " + topTransaction);

        // TODO 9: Mengecek apakah stack kosong menggunakan isEmpty()
        // Clue: Gunakan method isEmpty() untuk mengecek apakah stack kosong
        // Sintaks: boolean kosong = namaStack.isEmpty();
        System.out.println();
        if (transactionHistory._______()) {
          // Lengkapi method untuk cek stack kosong
            System.out.println("No transaction history.");
        } else {
            System.out.println("Transaction history is not empty. Size: " +
transactionHistory._______());
          // Lengkapi method untuk mendapatkan ukuran
        }

    }
}
```

DEMO

Task 1

Students must implement a mathematical expression processor using Stack. In programming and mathematics, expressions are commonly written in infix notation, such as

```
3 + 4 * 5
```

However, infix notation requires operator precedence rules and parentheses to determine the correct order of evaluation. To overcome this limitation, expressions can be converted into **`postfix notation`** (also known as **Reverse Polish Notation – RPN**), where operators appear after operands. You guys can read the details here.

Students must build a Java program that performs the following steps:

1. Infix Expression
   Input: Prompt the user to input a mathematical expression in infix notation. The expression may include:
   a. Integers
   b. Operators (+, -, *, /)
   c. Parentheses (and)

```
3 + 4 * 2 / ( 1 - 5 )
```

2. Infix to Postfix Conversion
   a. Convert the infix expression into postfix notation using the **Shunting-yard algorithm.**
   b. Use a Stack to handle operators and parentheses.
   c. The conversion must correctly respect operator precedence and associativity.
   Example Conversion:

```
Infix   : 3 + 4 * 2 / ( 1 - 5 )
Postfix : 3 4 2 * 1 5 - / +
```

3. Postfix Expression Evaluation
   Evaluate the resulting postfix expression using a Stack and push operands into the stack.
   When an operator is encountered:
   a. Pop the required operands
   b. Perform the operation
   c. Push the result back onto the stack
   Example Output :

```
Postfix Expression: 3 4 2 * 1 5 - / +
Result             : 1
```

4. Display the Output from the postfix

```
Masukkan ekspresi infix: 3 / 2 * 3 - ( 2 - 6 ) + 1
Infix    : 3 / 2 * 3 - ( 2 - 6 ) + 1
Postfix  : 3 2 / 3 * 2 6 - - 1 +
Result   : 9.5

Masukkan ekspresi infix: 2 + 3 * 6 / ( 2 - 5)
Infix    : 2 + 3 * 6 / ( 2 - 5)
Postfix  : 2 3 6 * 2 5 - / +
Result   : -4
```

Task 2

Students are required to build a console-based Java program that simulates the ticket service process at a railway station using Queue and Stack.
Students must implement the following features:

1. Queue-Based Ticket Service (FIFO)
   Use a Queue to store passengers waiting to be served. The program must accept user input to:
   a. Add passengers to the queue (*Create*)
   b. Display the current queue (*Read*)
   c. Passengers must be served in *First-In-First-Out (FIFO)*
2. Stack-Based Transaction History (LIFO)
   a. Use a Stack to store completed ticket transactions.
   b. Each served passenger must be pushed into the stack.
   c. The system must support undoing the last transaction using Stack (LIFO).

3. Main Class Implementation.
   Output like this:

```
=== Railway Ticket Service ===
1. Add Passenger
2. Display Queue
3. Serve Passenger
4. Undo Last Transaction
Choose menu: 1
Enter passenger name: Wira
Passenger added to queue.

=== Railway Ticket Service ===
1. Add Passenger
2. Display Queue
3. Serve Passenger
4. Undo Last Transaction
Choose menu: 1
Enter passenger name: Rapi
Passenger added to queue.
```

```
=== Railway Ticket Service ===
1. Add Passenger
2. Display Queue
3. Serve Passenger
4. Undo Last Transaction
Choose menu: 1
Enter passenger name: Zahra
Passenger added to queue.

=== Railway Ticket Service ===
1. Add Passenger
2. Display Queue
3. Serve Passenger
4. Undo Last Transaction
Choose menu: 2
Current Queue:
1. Wira
2. Rapi
3. Zahra
```

```
=== Railway Ticket Service ===
1. Add Passenger
2. Display Queue
3. Serve Passenger
4. Undo Last Transaction
Choose menu: 3
Serving passenger: Wira
Transaction saved.

=== Railway Ticket Service ===
1. Add Passenger
2. Display Queue
3. Serve Passenger
4. Undo Last Transaction
Choose menu: 2
Current Queue:
1. Rapi
2. Zahra
```

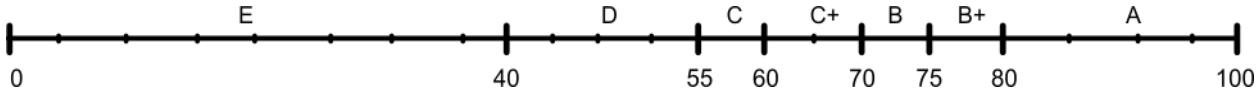
```
=== Railway Ticket Service ===
1. Add Passenger
2. Display Queue
3. Serve Passenger
4. Undo Last Transaction
Choose menu: 4
Undo transaction for passenger:

=== Railway Ticket Service ===
1. Add Passenger
2. Display Queue
3. Serve Passenger
4. Undo Last Transaction
Choose menu: 2
Current Queue:
1. Rapi
2. Zahra
3. Wira
```

Assessment

Grading Rubric

Assessment Aspect	Points (Total 100%)
Codelab	**Total 5%**
Successfully run smoothly	5%
Demo	**Total 95%**
Implement Postfix and Infiix	20%
Running smoothly task 1	5%
Implement Stack operations	15%
Implement Queue operations	15%
Running smoothly task 2	5%
Answer Questions	35%

Grading Scale



A = (81 - 100) → Sepuh

B+ = (75 - 80) → Very Good

B = (70 - 74) → Good

C+ = (60 - 69) → Fairly Good

C = (55 - 59) → Fair

D = (41 - 54) → Poor

E = (0 - 40) → Bro really...